

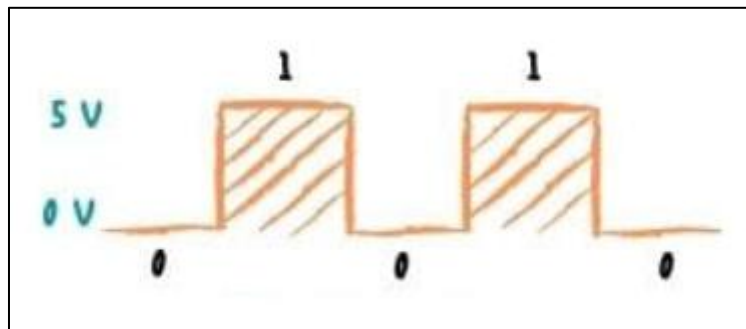
Lesson 11 DC Brushed Motor AB Quadrature Encoder Principles and Source Code Analysis

1. Preface

DC gear motor is a high-torque, high-load motor available in 8V and 12V variants, equipped with an encoder. This chapter focuses on controlling the operation of the JGA27-310 DC gear motor (referred to as the 310 motor) using an STM32 main control board.

Principle of AB quadrature encoder:

The rotary quadrature encoder is a digital IC (integrated circuit), so it uses binary code. Square waves indicate digital signals in high (1) and low (0) states.

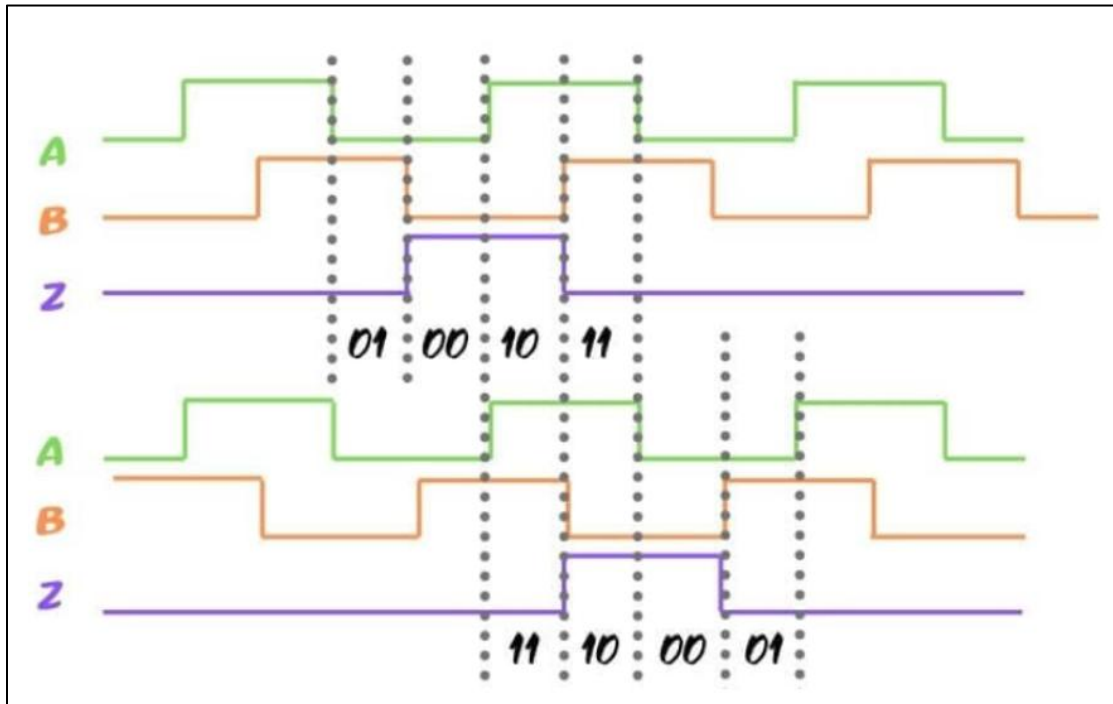


The operation of the encoder under 5V (using 5V as an example) power supply is illustrated in the above diagram, where the lines show low 0V (ground) and high 5V states.

Velocity determination: The number of pulses per revolution of the disk depends on the number of marks on its surface. This principle is the basic definition of encoder resolution. Therefore, two output signals represent the number of pulses in a specific time period. Using these values, the control system can determine angular velocity, distance, and acceleration.

Direction determination:

Two pins are placed sequentially. Thus, when simultaneously viewing the two waveform lines, you will notice a slight offset. Output channels A and B are specifically positioned on the disk at a certain distance, causing them to have a 90-degree phase offset on the waveform. The cycle is also divided into 4 equal parts.



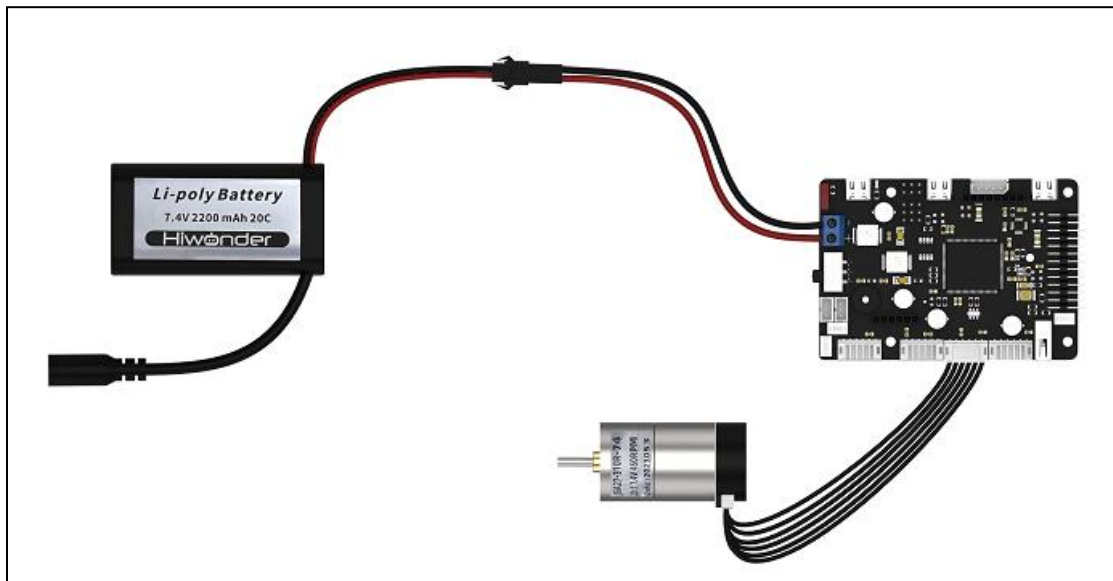
If the encoder moves forward (clockwise), line A is at the highest point (1), and channel B is at the lowest point (0). This results in combinations like 1 and 0, followed by 11, 00, and so on.

Starting from the same point in different directions (counterclockwise), you will see that this line will first be at the highest point (11), followed by combinations like 01, 00, and so on.

2. Hardware Connection

Required materials: 7.4V lithium battery pack, STM32 main control board, 310 motor.

The hardware connection diagram is as shown below:



Wiring method: Connect the 7.4V lithium battery and the DC power line of the STM32 main control board, then connect the 310 motor to the corresponding motor interface of the STM32 main control board according to the diagram.

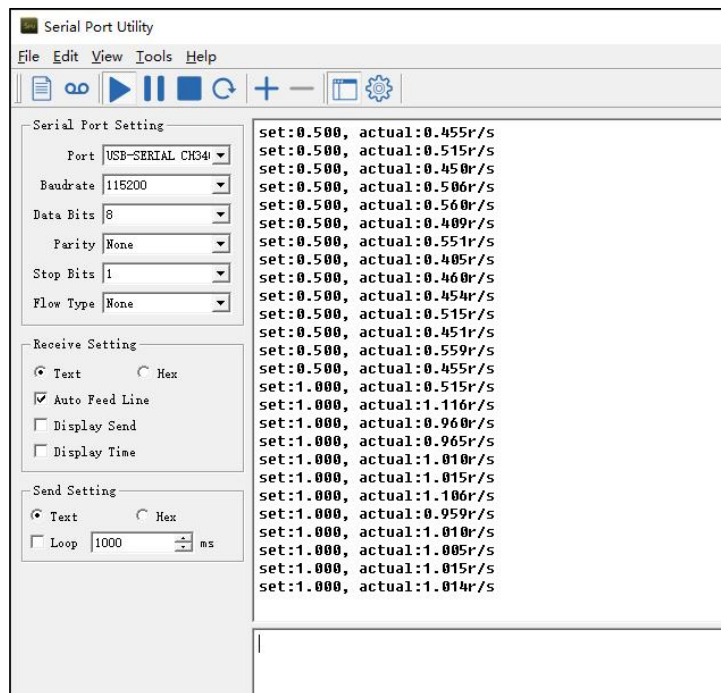
3. Program Download

The example routine for this content is named "EncoderMotor_PID_Demo": Specific instructions for program download can be found in "2.STM32 Development Fundamentals/Lesson 3 Project Compilation and Download" of the document. Simply replace the hex file mentioned in the document with the hex file generated from this example routine (EncoderMotor_PID_Demo.hex) for downloading.

4. Program Outcome

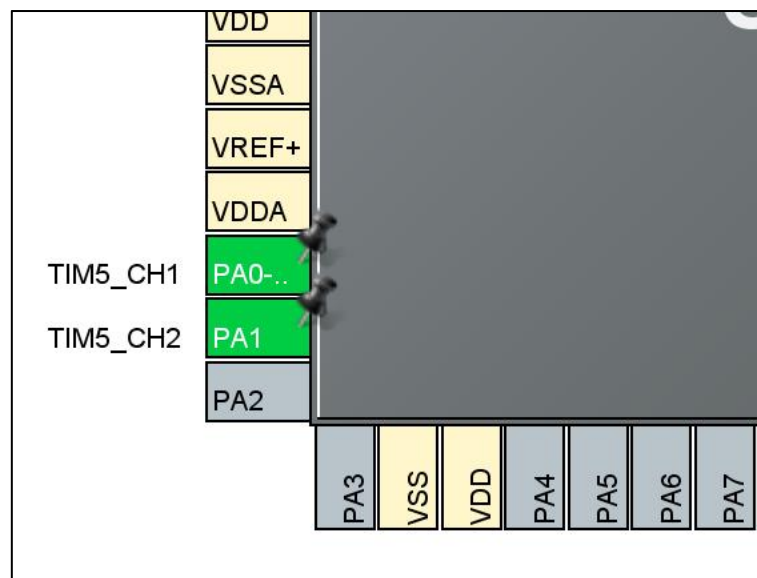
After powering on the STM32 control board, the motor rotates slowly for a period of time, then rotates quickly for another period of time, cycling repeatedly between these two operating states.

Open the serial port to check the actual and preset rotation speed of the current motor. The current speed continuously approaches the preset speed under the adjustment of the PID algorithm.



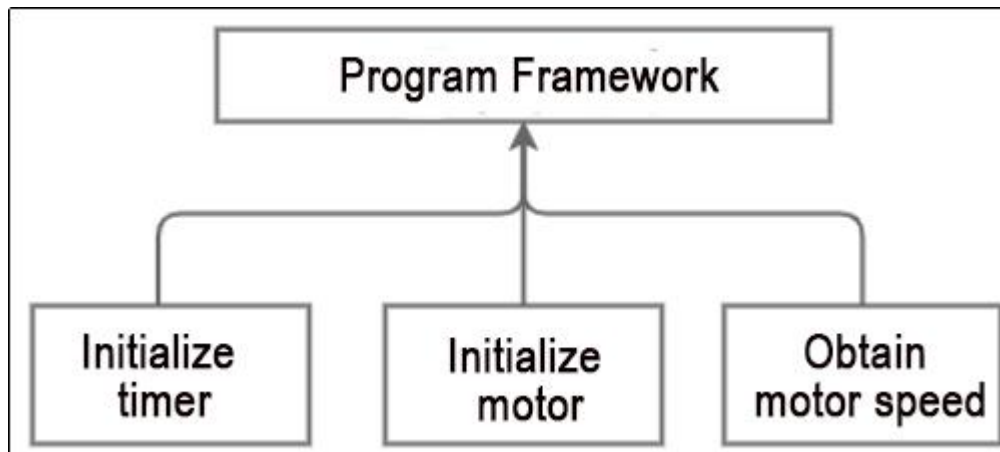
5. PIN Configuration

Using STM32CubeMX software to view pin information, during program execution, the external interrupt function will enable TIM5 timer loading. According to the diagram below, the corresponding chip pins are: PA0 and PA1.



6. Source Code Analysis

The program source code for this course is located in the "**Encoder_Demo**" folder. The program framework is designed based on the functionality as shown in the diagram below, with the main analysis conducted in the **main.c** file.



The program is divided into three parts: timer initialization, motor initialization, and motor speed acquisition.

By initializing the timer and motor, and acquiring the motor speed, the program verifies whether the AB phase control direction of the motor is correctly connected.

6.1 Initialize Timer

```
MX_TIM1_Init();  
MX_TIM5_Init();  
MX_TIM6_Init();
```

In the main function of the main.c file, three timers are initialized as shown in the diagram below: timers 1, 5, and 6 are configured with parameters. You can also refer to the tim.c file for details. These three timers have been configured through the interface in the STM32CubeMX software (it is recommended to use STM32CubeMX software for a more intuitive view).

▼ Counter Settings	
Prescaler (PSC - 16 bits value)	839
Counter Mode	Up
Counter Period (AutoReload Register - 16 bit...	999
Internal Clock Division (CKD)	No Division
Repetition Counter (RCR - 8 bits value)	0
auto-reload preload	Enable

Timer 1

▼ Counter Settings	
Prescaler (PSC - 16 bits value)	0
Counter Mode	Up
Counter Period (AutoReload Register - 32 bit...	60000
Internal Clock Division (CKD)	No Division
auto-reload preload	Enable

Timer 5

▼ Counter Settings	
Prescaler (PSC - 16 bits value)	83
Counter Mode	Up
Counter Period (AutoReload Register - 16 bits...	10000
auto-reload preload	Disable

Timer 6

6.2 Initialize Motor

Before controlling the motor operation, it is necessary to initialize the relevant parameters of the motor in the main function located in the main.c file.

motor_init();

```
void motors_init(void)
{
    for(int i = 0; i < 4; ++i) {
        motors[i] = LWMEM_CCM_MALLOC(sizeof( EncoderMotorObjectTypeDef));
        encoder_motor_object_init(motors[i]);
        motors[i]->ticks_overflow = 60000;
        motors[i]->ticks_per_circle = MOTOR_DEFAULT_TICKS_PER_CIRCLE;
        motors[i]->rps_limit = MOTOR_DEFAULT_RPS_LIMIT;
        motors[i]->pid_controller.set_point = 0.0f;
        motors[i]->pid_controller.kp = MOTOR_DEFAULT_PID_KP;
        motors[i]->pid_controller.ki = MOTOR_DEFAULT_PID_KI;
        motors[i]->pid_controller.kd = MOTOR_DEFAULT_PID_KD;
    }

    /* Motor 1 */
    motors[0]->set_pulse = motor1_set_pulse;
    __HAL_TIM_SET_COUNTER(&htim1, 0);
    __HAL_TIM_ENABLE(&htim1);
    __HAL_TIM_MOE_ENABLE(&htim1);

    /* encoder */
    __HAL_TIM_SET_COUNTER(&htim5, 0);
    __HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE(&htim5);
    HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL);

    /* Motor 2 */
    motors[1]->set_pulse = motor2_set_pulse;
    __HAL_TIM_SET_COUNTER(&htim1, 0);
    __HAL_TIM_ENABLE(&htim1);
    __HAL_TIM_MOE_ENABLE(&htim1);
}
```



```
// Speed update timer
__HAL_TIM_SET_COUNTER(&htim7, 0);
__HAL_TIM_CLEAR_IT(&htim7, TIM_IT_UPDATE);
__HAL_TIM_ENABLE_IT(&htim7, TIM_IT_UPDATE);
__HAL_TIM_ENABLE(&htim7);
```

From the above diagram, it can be observed that:

The initialized structure (encoder_motor_object_init):

```
encoder_motor_object_init(&motor1);
```

The parameters of motor1's structure are shown in the diagram, and the specific parameter contents can be referred to in the annotations. In the entire program, the parameter to pay attention to is rps, which is used to calculate the motor speed obtained.

```
/**
 * @brief Encoder motor object structure
 */
struct EncoderMotorObject {
    int64_t counter;          /**< @brief Total count value, assumes no overflow with 64-bit integer */
    int64_t overflow_num;     /**< @brief Overflow count value */
    int32_t ticks_overflow;   /**< @brief Count overflow value */
    float tps;                /**< @brief the frequency of the counter */
    float rps;                /**< @brief the output shaft speed in units of revolutions per second */
    int current_pulse;        /**< @brief The current PWM value of the output, with the sign indicating the direction of rotation */
    PID_ControllerTypeDef pid_controller; /**< @brief PID controller */

    /** Porting portable hardware interface */
    int32_t ticks_per_circle; /**< @brief The number of counts generated by one rotation of the motor output shaft, according to the actual situation of the motor */
    float rps_limit;          /**< @brief The maximum speed of the motor, which is usually set to a slightly lower value than the speed at 100% PWM duty cycle to ensure control */

    /**
     * @brief set motor speed -1000-1000
     * @param encoder motor object
     * @param the new PWM value, ranging from -1000-1000, corresponding to 0-100% speed in both forward and reverse directions
     * @retval None
     */
    void (*set_pulse)(EncoderMotorObjectTypeDef *self, int pulse);
};
```

Using Timer 1 to generate PWM signals, as shown in the diagram below:

```
__HAL_TIM_SET_COUNTER(&htim1, 0);
__HAL_TIM_ENABLE(&htim1);
__HAL_TIM_MOE_ENABLE(&htim1);
```

Using TIM5 to control the encoder of the encoder motor for acquisition and startup, as shown below:

```
/* encoder */
__HAL_TIM_SET_COUNTER(&htim5, 0);
__HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);
__HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);
__HAL_TIM_ENABLE(&htim5);
HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL);
```

Using TIM7 to calculate the encoder value once to obtain the speed, as shown below:

```
// Speed update timer
__HAL_TIM_SET_COUNTER(&htim7, 0);
__HAL_TIM_CLEAR_IT(&htim7, TIM_IT_UPDATE);
__HAL_TIM_ENABLE_IT(&htim7, TIM_IT_UPDATE);
__HAL_TIM_ENABLE(&htim7);
```

Note: When initializing the motor, it is necessary to change the PID parameters, speed limit, rotation count value, and other parameters according to the different motor models in the functions. Please refer to the code comments in “6.2 Initialize Motor” for details. To facilitate the usage, we introduced the "set_chassis_type" function and provided corresponding motor macro definitions.

When using it, you do not need to modify the internal of the "motor_init" function. After calling it, you can use the "set_chassis_type" function to override the motor parameters of the specified motor model into the configuration structure. Please refer to the definition inside the function for which motor model each macro definition corresponds to.

In this example, the JGB37 motor is used as an example. Call

“set_chassis_type(CHASSIS_TYPE_TANKBLACK)” in the app task.

You can modify the input parameters to specify the motor model you are using.

6.3 Adjust Speed

The program is located in the function app_task_entry in app.c, as shown in the figure below:

```
for(;;) {
    //encoder_motor_set_speed() Set rotation speed of the servo with PID algorithm. Parameter 1: motor object; parameter 2: set motor speed (r/s). After compiling,
    encoder_motor_set_speed(motors[0], 1.0); /* set target rotation speed to 1.0 */
    for(int i = 0; i < 100; ++i) {
        printf("set:%0.3f, actual:%0.3f/s\r\n", motors[0]->pid_controller.set_point, motors[0]->rps); // print the set speed and measured speed
        osDelay(100); // delay for 100ms
    }
    encoder_motor_set_speed(motors[0], 0.5); /* set target rotation speed to 0.5 */
    for(int i = 0; i < 100; ++i) {
        printf("set:%0.3f, actual:%0.3f/s\r\n", motors[0]->pid_controller.set_point, motors[0]->rps); // print the set speed and measured speed
        osDelay(100); // delay for 100ms
    }
}
```

Use the function encoder_motor_set_speed to set the PID target value (desired speed) of motor 1 to 1 (rev/s). It will not be executed immediately. In fact, the execution is completed in the interrupt of timer 7.

The detailed definition of the encoder_motor_set_speed function is shown below:

```
/**
 * @brief Encoder motor setting PID control target speed
 * @param self self Encoder motor object pointer
 * @param rps Target speed, in revolutions per second
 * @retval None.
 */
void encoder_motor_set_speed(EncoderMotorObjectTypeDef *self, float rps)
{
    rps = rps > self->rps_limit ? self->rps_limit : (rps < -self->rps_limit ? -self->rps_limit : rps); /* Limiting the speed */
    self->pid_controller.set_point = rps; /* Set PID controller objectives */
}
```


In this function, after the preset speed is limited, it is only assigned to the speed variable member of the structure after being limited. It can be seen that there is another function responsible for implementing the target speed. Here, we only provide the target data.

Every 10000us, timer 7 overflows and enters an interrupt, where we perform further operations on the set parameters. The detailed definition is as follows:

```
void TIM7_IRQHandler(void)
{
    /* USER CODE BEGIN TIM7_IRQn 0 */
    extern EncoderMotorObjectTypeDef *motors[4];
    if(__HAL_TIM_GET_FLAG(&htim7, TIM_FLAG_UPDATE) != RESET) {
        __HAL_TIM_CLEAR_FLAG(&htim7, TIM_FLAG_UPDATE);
        encoder_update(motors[0], 0.01, __HAL_TIM_GET_COUNTER(&htim5));
        encoder_update(motors[1], 0.01, __HAL_TIM_GET_COUNTER(&htim2));
        encoder_update(motors[2], 0.01, 60000 - __HAL_TIM_GET_COUNTER(&htim4));
        encoder_update(motors[3], 0.01, __HAL_TIM_GET_COUNTER(&htim3));
        for(int i = 0; i < 4; ++i) {
            encoder_motor_control(motors[i], 0.01);
        }
    }
    /* USER CODE END TIM7_IRQn 0 */
    /* USER CODE BEGIN TIM7_IRQn 1 */

    /* USER CODE END TIM7_IRQn 1 */
}
```

First, for each motor, the encoder_update function is used once to calculate the incremental count recorded by the encoder (motor 1's encoder is completed by timer 5 in encoder mode) since the last time it entered the interrupt.

For motor 1, the parameter motors[0] provides the total count value of the last time, the parameter __HAL_TIM_GET_COUNTER(&htim5) reads the current count value of timer 5, and provides the count cycle of timer 7 (the time interval between each timer interrupt trigger) 10000us=0.01s.

Assuming we know the count increment of one revolution of the motor (see the manufacturer's data manual), and we know the count increment in 0.01s, we can naturally know how many revolutions the motor has turned in 0.01s, and thus know the current speed.

Then call the encoder_motor_control function to calculate the PID output parameters based on the preset PID model, current speed, and target speed

(ours), and write the newly calculated PWM duty cycle into timer 1.

At this step, the logic chain for adjusting speed is considered complete.

Note: This section of the program involves the use of PID, which will be explained in detail in the next section. Here, we only provide a brief summary. The complete definition of the `encoder_motor_control` function is shown below:

The complete definition of the `encoder_motor_control` function is shown below:

```
void encoder_motor_control(EncoderMotorObjectTypeDef *self, float period)
{
    float pulse = 0;
    pid_controller_update(&self->pid_controller, self->rps, period); /* Update PID controller */
    pulse = self->current_pulse + self->pid_controller.output; /* Calculate new PWM values */

    /* Limit the output PWM value according to the timer setting. In this example, the duty cycle of the timer set from 0-100 corresponds to 0-1000 */
    pulse = pulse > 1000 ? 1000 : pulse;
    pulse = pulse < -1000 ? -1000 : pulse;

    self->set_pulse(self, pulse > 250 || pulse < -250 ? 0 : pulse); /* Set a new PWM value and limit the minimum PWM value. If the PWM is too small, the motor will stop */
    self->current_pulse = pulse; /* Record new PWM values */
}
```

6.4 Obtain Speed

Using a while loop to obtain the motor speed and print it via serial port:

```
while (1) {
    printf("counter:%lld, speed:%0.3fr/s\r\n", motor1.counter, motor1.rps);
    HAL_Delay(100);
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
```

From the above diagram, it can be seen that the motor speed can be obtained through the `motor.rps` parameter. By printing the current motor rotation speed via serial port, it is possible to verify the accuracy of the motor's AB phase data. To ensure that the motor speed is not lost due to being too fast, a 100-millisecond delay is implemented using the `HAL_Delay` function.

Note: The main focus of the source code analysis is on the key structure of the DC motor drive program. This helps users quickly understand the structure and content of the code. Users can refer to the code and corresponding comments for detailed learning and reference.
